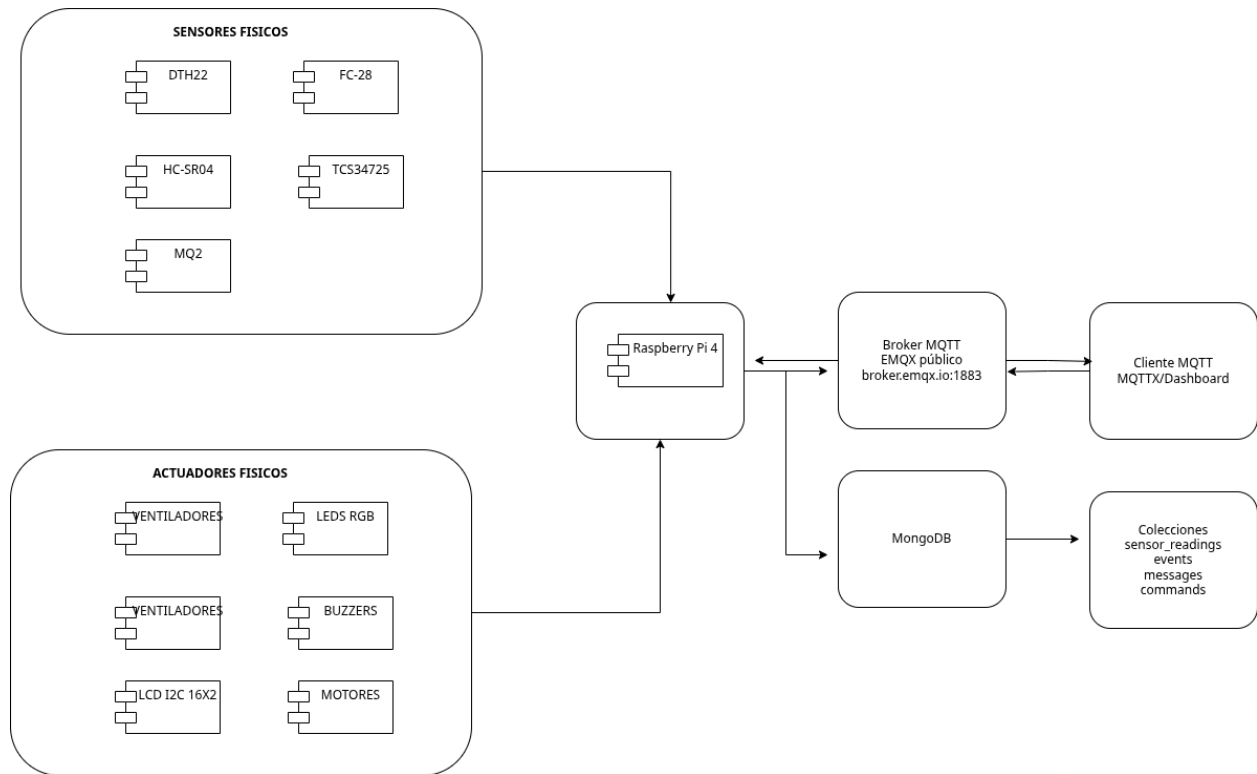


Informe técnico del sistema de Nave Starwars

1. Resumen de la arquitectura

La solución se basa en una Raspberry Pi 4 que centraliza la lectura de sensores, la ejecución de reglas de decisión, el control de actuadores, la visualización local en LCD, la publicación/recepción de mensajes vía MQTT y la persistencia de lecturas y eventos en MongoDB. El software se organizó en módulos por responsabilidad: hardware, lógica, servicios y threads.

Diagrama de Arquitectura



2. Hardware y asignación principal de GPIO

Elemento	GPIO / Bus	Observación
DHT22	GPIO4	Temperatura
HC-SR04	GPIO17 / GPIO27	TRIG / ECHO con divisor
MQ-2	SPI -> MCP3208 CH0	Sensor de gas analógico
FC-28	SPI -> MCP3208 CH1	Humedad de suelo
TCS34725	I2C	Sensor de color y luz
LCD 16x2	I2C	Pantalla informativa
Relé ventiladores	GPIO24	Control de ambos ventiladores
Bus RGB (R/G/B)	GPIO21 / GPIO16 / GPIO26	8 LED RGB en paralelo lógico
LED amarillo de estado	GPIO20	Indicador de alerta
Buzzer	GPIO23	Alertas sonoras
Servo SG90	GPIO18	Apertura/cierre de compuerta
28BYJ-48	GPIO5, 6, 13, 19	Torreata

3. Stack tecnológico

Componente	Tecnología
Lenguaje	Python 3
Hardware	Raspberry Pi 4
Comunicación	MQTT (EMQX broker)
Base de datos	MongoDB
Concurrencia	threading
Sensores	DHT22, MQ-2, FC-28, HC-SR04, TCS34725
Actuadores	Relé, LEDs RGB, buzzer, LCD

4. Topics MQTT del proyecto

nave/sensores/gas/gp3

nave/sensores/proximidad/gp3

nave/sensores/color/gp3

nave/sensores/ambiente/gp3

nave/sensores/suelo/gp3

nave/estado/gp3

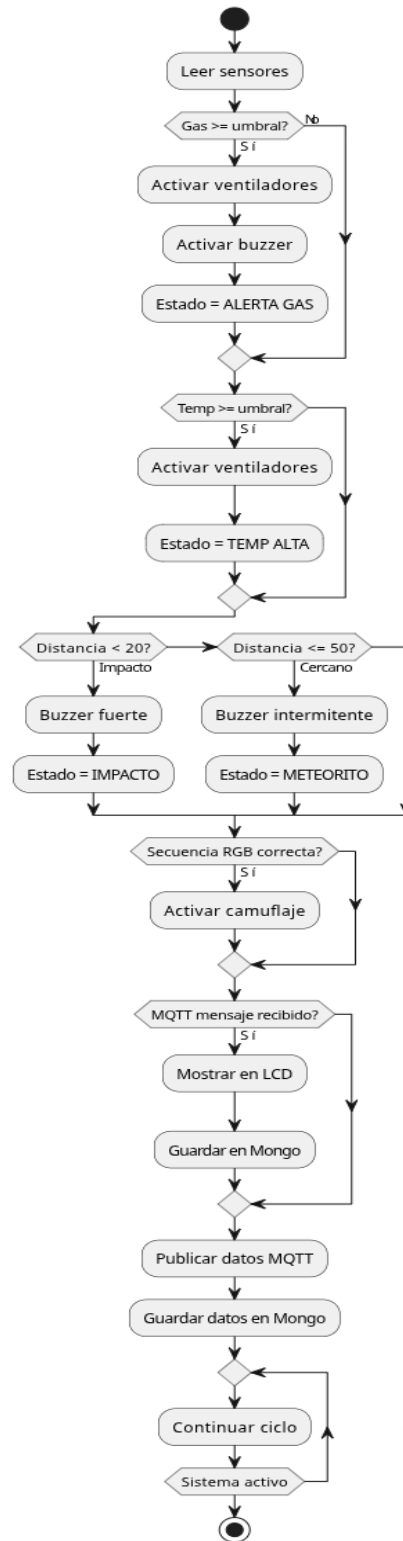
nave/control/mensajes/gp3

nave/alertas/criticas/gp3

5. Flujo de la lógica de respuesta automática

La lógica corre continuamente en hilos. Primero se toman lecturas; después se evalúan reglas; finalmente se actualizan salidas, se publica por MQTT y se almacena en MongoDB.

Flujo Lógico - Sistema Automatizado



6. Regla de decisión principal

- Si el MQ-2 supera el umbral configurado, se activa alerta de gas y se energiza el relé de ventiladores.
- Si la temperatura del DHT22 supera el umbral, también se energizan los ventiladores.
- Si el sensor ultrasónico detecta cercanía, el buzzer responde por niveles.
- Si el TCS34725 detecta la secuencia rojo -> amarillo -> azul, se activa camuflaje por un tiempo determinado y los LED RGB replican el color detectado.
- Todas las lecturas se muestran localmente en el LCD y también se publican al broker MQTT.
- Las lecturas y eventos relevantes se almacenan en MongoDB para evidencia e historial.

7. Estructura modular del software

El código se dividió en módulos para facilitar mantenimiento, pruebas y crecimiento del proyecto.

```
ARQI1-G3/  
├── app.py  
├── config.py  
├── shared_state.py  
├── hardware/  
├── services/  
├── logic/  
└── threads/
```

8. Colecciones MongoDB

- sensor_readings: lecturas periódicas de sensores.
- events: eventos de alerta, camuflaje y cambios de estado.
- messages: mensajes recibidos por MQTT para el LCD.
- commands: comandos entrantes por topics de control.